CareTrack

Software Requirements Specification (SRS)

Project Name: CareTrack
Team Name: Team2
Document Type: Software Requirements Specification
GitHub Repository: GitHub - ernkrkt10/akilli-takip-sistemi

1. Introduction

1. **Purpose:** The purpose of this SRS document is to provide a clear and detailed description of the functional and non-functional requirements of CareTrack. This document defines what the system is expected to do, how it should perform, and the limitations of it. It also helps ensure that the final system meets the defined requirements and user needs.
2. **Document Conventions:**

   FR: Functional Requirement

   NFR: Non-Functional Requirement

   US: User Story

   UC: Use Case

3. **Intended Audience:**

   Developers: To understand the functions and technical requirements of the system.

   Testers: To prepare and perform tests based on the requirements.

   Users: To understand the features and services provided by CareTrack.

   Project Managers: To follow the project scope, requirements, and planned features.

4. **Project Scope:** CareTrack is a web application that helps users manage their doctor appointments and personal medication records in one place.

   The final version of the system will allow users to:

   - Create an account, sign in, and sign out.
   - Add, view, edit, and delete doctor appointments.

- Save medication plans based on instructions received from a health professional.
- View planned medication times.
- Record whether a planned medication dose was taken, skipped, or postponed.
- Review past appointment and medication tracking records.
- View upcoming appointments and planned medication times on a dashboard.

CareTrack will not provide:

Medical advice, diagnose health conditions, recommend medicines, calculate medication doses, or change treatment instructions. It will not connect to hospital systems, pharmacies, or national health records. Automatic sharing with family members or healthcare staff, mobile applications, wearable devices, payment services, video appointments, and guaranteed background notifications are also outside the scope.

5. **References:** *CareTrack – Software Development Plan (D1)*
   https://github.com/ernkrkt10/akilli-takip-sistemi.git

2. Overall Description

1. **Product Perspective:** CareTrack is a standalone web application designed to help users manage their personal doctor appointments and medication records.

   Users will access the application through a web browser. The application will store information entered by each user in its own database. Users will be able to manage their records without using separate paper notes or calendar applications.

   The first version will not integrate with external healthcare systems, hospital databases, pharmacy services, or national health records. It will work as an independent application.

2. **Product Functions:**
   - **User Registration:** Allow new users to create an account.

- **User Authentication:** Allow registered users to sign in and sign out.
- **Dashboard:** Display upcoming appointments and planned medication times.
- **Appointment Management:** Allow users to add, view, edit, and delete doctor appointments.
- **Medication Plan Management:** Allow users to save, view, edit, and delete their medication plans.
- **Medication Tracking:** Allow users to record a medication status as Taken, Skipped, or Postponed.
- **Appointment History:** Allow users to review their saved past appointments.
- **Medication History:** Allow users to review their previous medication tracking records.
- **Input Validation:** Check required fields and display error messages when information is missing or invalid.
- **Record Privacy:** Ensure that users can only access their own private records.

3. User Classes and Characteristics:

Main User / Patient

The main user is a person who wants to manage personal doctor appointments and medication records.

- The user may have basic computer and internet skills.
- The user needs a simple and clear interface.
- The user should be able to enter and update information easily.
- The user needs to access personal records privately.
- The user should be able to review previous appointments and medication tracking records.

4. Operating Environment:

CareTrack will operate as a web application.

- Operating Systems: Windows, macOS, and other operating systems that support modern web browsers.
- Web Browsers: Google Chrome and Microsoft Edge.
- Devices: Desktop computers, laptops, and mobile devices with a supported web browser.
- Internet Connection: An internet connection or the local development connection will be required to access the running application.

- Development Environment: The application will be developed and tested on team members' computers.

5. **Design and Implementation Constraints:**
   - **Technology Stack:** Must use Python/Django for the web backend, standard HTML/CSS Django templates for the interface, and SQLite for data storage
   - **Resource & Budget:** Academic zero-budget limit; relies exclusively on free built-in Python/Django packages, standard web-browser access, and local execution (localhost) without paid infrastructure or third-party paid APIs.
   - **Data Privacy & Test Scope:** Implements Django user session isolation so accounts only access their own logs.
   - **Medical Disclaimer & Non-Clinical Scope:** System acts strictly as a manual record-keeping log; it does not calculate dosages, suggest treatments, or provide clinical diagnoses.
   - **System Boundaries:** No integrations with external hospital/pharmacy networks, external messaging services, or mobile/wearable hardware; restricted entirely to a standard browser-based web application.

Assumptions and Dependencies:

- Users will enter their appointment details correctly.
- Users will enter medication information based on instructions received from a qualified health professional.
- Users will have access to a supported web browser and the running application.
- Users will sign in to log taken or skipped doses.
- User devices will maintain accurate local system time for schedule calculations.
- The local host environment runs Python and Django without server crashes.

Functional Requirements

This section describes the functions that CareTrack must provide.

User Stories:

Epic 1: User Account Management

US-01: Register New Account As a Main User, I want to create an account using my email address and a password, so that I can securely manage my health tracking records.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am on the registration page, When I enter a valid email and a secure password, Then the system creates my account successfully.
  - Given I have created an account, When the registration is successful, Then my password is saved securely using Django's built-in password-hashing feature.

US-02: Sign In As a Main User, I want to sign in to my account with my correct login details, so that I can access my private dashboard and records safely.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am on the login page, When I enter my correct email and password, Then I am authenticated and redirected to my dashboard.
  - Given I am not signed in, When I attempt to access private pages via URL, Then the system blocks my access and redirects me to the login page.

US-03: Sign Out As a Main User, I want to sign out of my account, so that I can ensure my medical records remain private when I step away from the device.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am currently signed in, When I click the sign-out button, Then my session is terminated.
  - Given I have signed out, When I try to use the back button to view my records, Then the system requires me to log in again.

Epic 2: Dashboard

US-04: View Upcoming Appointments on Dashboard As a Main User, I want to see my upcoming doctor appointments on the dashboard immediately after signing in, so that I don't miss any scheduled visits.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I have signed in successfully, When the dashboard loads, Then a list of my upcoming appointments is displayed.
  - Given I am viewing the dashboard, When the list populates, Then I only see appointments belonging to my own account.

US-05: View Planned Medication Times on Dashboard As a Main User, I want to view my planned medication times on the dashboard, so that I can quickly check when my next dose is due.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I have signed in successfully, When the dashboard loads, Then my planned medication times are displayed alongside my appointments.

Epic 3: Appointment Management

US-07: Create an Appointment As a Main User, I want to add an appointment by entering its date, time, doctor/clinic, and reason, so that I can keep track of it.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am on the Add Appointment form, When I fill in all required fields and submit, Then the record is saved and a success message is displayed.
  - Given I am adding an appointment, When I enter an invalid appointment date, Then the system displays an error message and prevents saving.

US-08: View Saved Appointments As a Main User, I want to see a dedicated list of all my saved appointments, so that I can review my medical calendar.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I navigate to the Appointments page, When the page loads, Then it displays all appointments saved by my account.

US-09: Edit Appointment Details As a Main User, I want to edit an existing appointment, so that I can update the date or time if my doctor reschedules.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am viewing my appointments, When I select edit and update the time, Then the system saves the new details.
  - Given I am editing a record, When the save process fails due to a system issue, Then the system displays an error message and does not display a false success message.

US-10: Delete an Appointment As a Main User, I want to delete an appointment, so that I can remove cancelled visits from my records.

- **Priority:** Should Have
- **Acceptance Criteria:**
  - Given I select an appointment to delete, When I trigger the delete action, Then the system asks for confirmation before deleting to prevent accidental deletion.
  - Given the confirmation prompt is visible, When I confirm the deletion, Then the appointment is permanently removed from my list.

Epic 4: Medication Plan Management

US-11: Save a Medication Plan As a Main User, I want to save a medication name, clinician instructions, and planned times, so that my personal treatment schedule is accurately recorded.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am on the Add Medication Plan page, When I enter the required name and instructions, Then the system saves the plan successfully.
  - Given I am adding a medication plan, When I input a planned time using an invalid time format, Then the system displays an error message and prevents saving.

US-12: Edit a Medication Plan As a Main User, I want to edit my saved medication plans, so that I can update the instructions or times if my health professional changes them.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I am viewing my Medication Plans, When I select edit and modify the

clinician instructions, Then the updated plan is saved correctly.

US-13: Delete a Medication Plan As a Main User, I want to delete a medication plan, so that I can remove medicines I no longer need to take from my active list.

- **Priority:** Should Have
- **Acceptance Criteria:**
  - Given I click to delete a medication plan, When the system prompts for confirmation, Then the record is only removed if I explicitly confirm.

Epic 5: Medication Tracking and History

US-14: Record Medication Status As a Main User, I want to record my planned medication dose as Taken, Skipped, or Postponed, so that I maintain an accurate log of my adherence to the plan.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I have an active planned time, When I select a status (Taken, Skipped, or Postponed), Then the system saves the status along with the current date and time.
  - Given I attempt to record a status, When I select Save, Then the system displays a success or error response within 3 seconds.

US-15: Review Tracking History As a Main User, I want to review my previous medication tracking records and past appointments, so that I can discuss my history accurately during my next doctor visit.

- **Priority:** Must Have
- **Acceptance Criteria:**
  - Given I navigate to the History page, When the page loads, Then it displays a chronological log of my past medication tracking records.
  - Given I am on the History page, When I check my past appointments, Then only appointments belonging to my account are visible.

Use Cases:

Use Case 1: Manage Doctor Appointments (UC-01)

- **Actor:** Main User / Patient
- **Preconditions:** The user is successfully signed into their CareTrack account.
- **Main Flow:**

  1. The user navigates to the "Appointments" section from the Dashboard.
  2. The user selects the option to add or edit an appointment.
  3. The user inputs the date, time, doctor or clinic, and the reason for the visit.
  4. The user submits the form.
  5. The system validates the provided information (checks for empty required fields and valid date formats).
  6. The system saves the appointment to the SQLite database, linked specifically to the user's account.
  7. The system displays a success message confirming the action**.**

- **Alternative Flows:**

  - *5a. Invalid Input:* If a required field is empty or the date is invalid, the system

rejects the submission, does not save the record, and displays a clear error message prompting the user to correct the information.

- **Postconditions:** The appointment is saved and is now visible on the user's Dashboard and Appointment History pages.

Use Case 2: Record Medication Tracking Status (UC-02)

- **Actor:** Main User / Patient
- **Preconditions:** The user is signed in and has at least one active Medication Plan saved in the system.
- **Main Flow:**

  1. The user navigates to the "Medication Tracking" feature.
  2. The system displays the user's planned medication times.
  3. The user selects a specific medication dose.
  4. The user chooses a tracking status from the options: Taken, Skipped, or Postponed.
  5. The system saves the selected tracking status along with the exact record date and time.
  6. The system displays a success message to the user.

- Alternative Flows:

  - *5a. System Failure:* If the database or application is temporarily unavailable and cannot save the record, the system displays an appropriate error message and explicitly does not show a success message.

- Postconditions: The dose status is permanently added to the Dose Status Record database table and can be reviewed in the Medication History section.

User Account Management:

- **FR-1:** The system shall allow a new user to create an account using an email address and password.
- **FR-2:** The system shall allow a registered user to sign in using the correct login details.
- **FR-3:** The system shall allow a signed-in user to sign out.
- **FR-4:** The system shall prevent users who are not signed in from accessing private pages.

Dashboard:

- **FR-5:** The system shall display a dashboard after a user signs in.
- **FR-6:** The dashboard shall display the user's upcoming appointments.

- **FR-7:** The dashboard shall display the user's planned medication times.
- **FR-8:** The system shall display a clear message when there are no upcoming appointments or planned medication times.

Appointment Management:

- **FR-9:** The system shall allow a user to create an appointment by entering its date, time, doctor or clinic, and reason.
- **FR-10:** The system shall display the appointments saved by the signed-in user.
- **FR-11:** The system shall allow a user to edit their own appointment details.
- **FR-12:** The system shall ask for confirmation before deleting an appointment.

Medication Plan Management:

- **FR-13:** The system shall allow a user to save a medication name, instructions, and planned times.
- **FR-14:** The system shall display the medication plans saved by the signed-in user.
- **FR-15:** The system shall allow a user to edit their own medication plans.
- **FR-16:** The system shall ask for confirmation before deleting a medication plan.

Medication Tracking and History:

- **FR-17:** The system shall allow a user to record a planned medication dose as Taken, Skipped, or Postponed.
- **FR-18:** The system shall save the selected medication status with its date and time.
- **FR-19:** The system shall display the signed-in user's previous medication tracking records.
- **FR-20:** The system shall display the signed-in user's past appointment records.

Input Validation and Record Privacy:

- **FR-21:** The system shall display an error message when a required field is empty or an appointment date is invalid.
- **FR-22:** The system shall display a success message after a record is saved successfully.
- **FR-23:** The system shall ask the user to confirm before deleting a record.
- **FR-24:** The system shall only allow users to view, edit, or delete records that belong to their own accounts.

Non-Functional Requirements

Performance Requirements:

- **NFR-1:** The dashboard and main list pages shall load within 3 seconds when tested on the team's development computer with up to 100 sample records.
- **NFR-2:** After a user selects Save, the system shall display a success or error response within 3 seconds under normal local testing conditions.
- **NFR-3:** The application shall use the development computer's resources efficiently and shall not require special hardware to run the first version.

Security Requirements:

- **NFR-1:** The system shall require users to sign in before displaying private appointment or medication records.
- **NFR-2:** The system shall check record ownership before allowing a user to view, edit, or delete a record.
- **NFR-3:** User passwords shall be stored using Django's built-in password-hashing feature and shall not be stored as plain text.
- **NFR-4:** The application shall not display one user's private records to another user.
- **NFR-5:** The development and demonstration versions shall use fictional user and medication information only.
- **NFR-6:** Passwords and secret keys shall not be stored in the public project repository.

Usability Requirements:

- **NFR-1:** The application shall use clear labels and consistent menu names for Dashboard, Appointments, Medication Plans, and History.
- **NFR-2:** At least 4 out of 5 test users shall be able to add an appointment and save a medication plan without help.
- **NFR-3:** Forms shall display clear error messages when required information is missing or invalid.
- **NFR-4:** The main pages shall use readable text, simple navigation, and consistent page layouts.
- **NFR-5:** The system shall display a confirmation message after a record is saved successfully.
- **NFR-6:** The system shall ask for confirmation before deleting a record to help

prevent accidental deletion.

Reliability and Availability:

- **NFR-1:** At least 19 out of 20 valid record-saving attempts shall complete successfully during the team's functional tests.
- **NFR-2:** The system shall display an error message if a record cannot be saved. It shall not display a success message when the saving process fails.
- **NFR-3:** The team shall test the create, view, edit, and delete operations before the final demonstration.
- **NFR-4:** The team shall keep a copy of the project database and sample data during development so that test records can be restored if needed.
- **NFR-5:** The application shall display an appropriate error message when the database or application is temporarily unavailable.

Maintainability Requirements:

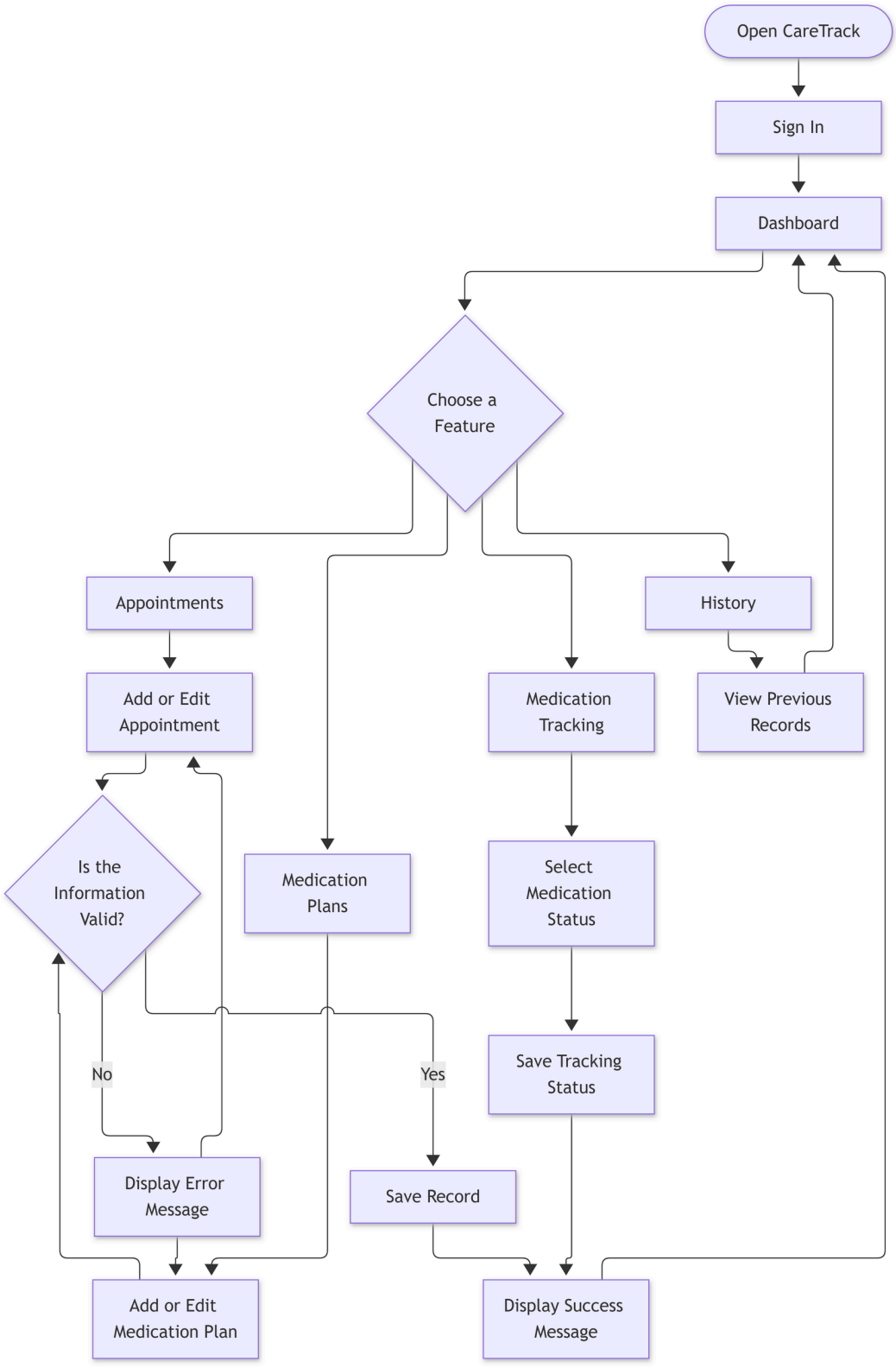
- **NFR-1:** The project code shall be organised into clearly named Django application areas and files.
- **NFR-2:** The team shall keep setup and testing instructions in the project's README file.
- **NFR-3:** Each completed main feature shall have at least one related test or documented manual test case.
- **NFR-4:** Code changes shall be reviewed by another team member before they are merged into the main branch.
- **NFR-5:** The project shall use clear names for database models, views, forms, and templates so that team members can understand and update the code.

Compatibility Requirements:

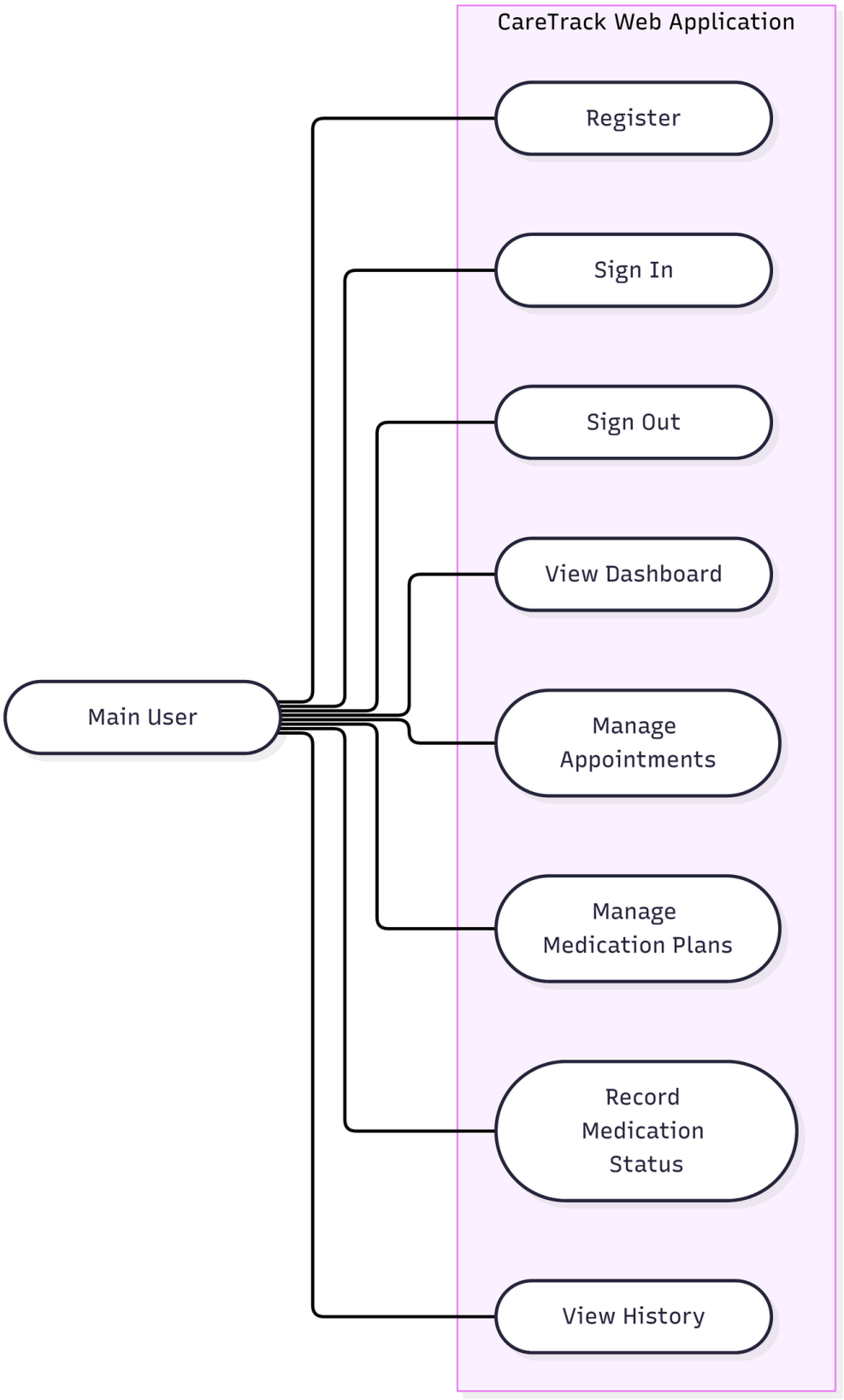
- **NFR-1:** The web application shall be tested using the current versions of Google Chrome and Microsoft Edge.
- **NFR-2:** The main pages shall remain readable at common desktop and mobile screen widths.
- **NFR-3:** The first version shall be accessible through a modern web browser and shall not require a separate mobile application.
- **NFR-4:** The application shall support operating systems that can run the supported web browsers.
- **NFR-5:** The application shall use standard HTML and CSS features supported by the selected browsers.

System Models

Use Case Diagram:



User Flow Diagram:


CareTrack Web Application
Register
Sign In
Sign Out
View Dashboard
Main User
Manage Appointments
Manage Medication Plans
Record Medication Status
View History

Data Requirements

Main Data Entities:

Entity	Information Stored	Basic Validation
User	Email address, password (stored securely by the authentication system).	Email is required and must have a valid format. Password is required.
Appointment	Owner, date, time, doctor or clinic, reason for visit.	Date, time, and doctor or clinic are required. Date must be valid.
Medication Plan	Owner, medication name, clinician instructions, planned times.	Medication name and instructions are required. Planned times must use a valid time format.
Dose Status Record	Owner, related medication plan, status, record date and time.	Status must be Taken, Skipped, or Postponed. Date and time are required.

External Interfaces

User Interface: Users access CareTrack through a web browser. The system provides registration and sign-in pages, a dashboard, appointment forms, medication plan forms, and a history page. Forms display clear success and error messages.

External Services: CareTrack does not use external APIs or third-party services. All planned features work within the application and its own database.